Assignment: Earley Parser Implementation

Name: Ekansh Goyal
Roll Number: 2023114007
Code Repository: GitHub Link

Part 1: Simulating the Earley Algorithm

Objective: Trace the Earley parser's execution on the input string "time flies like an arrow" using the initial grammar, and present the resulting chart.

The final state of the parser's chart is detailed below. Any state that has been fully completed is prefixed with an asterisk (*).

Final Parser Chart

```
-- Initial State (Position 0) --
[0,0] S -> . NP VP
[0,0] NP -> . N
[0,0] NP -> . Det N
[0,0] NP -> . N N
[0,0] N -> . time
[0,0] N -> . flies
[0,0] N -> . arrow
[0,0] Det -> . a
[0,0] Det -> . an

-- Processing Word 1: 'time' --
* [0,1] N -> time .
* [0,1] NP -> N .
[0,1] NP -> N . N
[0,1] S -> NP . VP
[1,1] N -> . time
[1,1] N -> . flies
[1,1] N -> . arrow
[1,1] VP -> . V NP
[1,1] VP -> . V ADVP
[1,1] V -> . flies
[1,1] V -> . like

-- Processing Word 2: 'flies' --
* [1,2] N -> flies .
* [1,2] V -> flies .
* [0,2] NP -> N N .
[1,2] VP -> V . NP
[1,2] VP -> V . ADVP
[0,2] S -> NP . VP
[2,2] NP -> . N
[2,2] NP -> . Det N
[2,2] NP -> . N N
[2,2] ADVP -> . ADV NP
[2,2] VP -> . V NP
```

```
[2,2] VP -> . V ADVP
[2,2] N -> . time
[2,2] N -> . flies
[2,2] N -> . arrow
[2,2] Det -> . a
[2,2] Det -> . an
[2,2] ADV -> . like
[2,2] V -> . flies
[2,2] V -> . like

-- Processing Word 3: 'like' --
* [2,3] ADV -> like .
* [2,3] V -> like .
[2,3] ADVP -> ADV . NP
[2,3] VP -> V . NP
[2,3] VP -> V . ADVP
[3,3] NP -> . N
[3,3] NP -> . Det N
[3,3] NP -> . N N
[3,3] ADVP -> . ADV NP
[3,3] N -> . time
[3,3] N -> . flies
[3,3] N -> . arrow
[3,3] Det -> . a
[3,3] Det -> . an
[3,3] ADV -> . like

-- Processing Word 4: 'an' --
* [3,4] Det -> an .
[3,4] NP -> Det . N
[4,4] N -> . time
[4,4] N -> . flies
[4,4] N -> . arrow

-- Processing Word 5: 'arrow' --
* [4,5] N -> arrow .
* [3,5] NP -> Det N .
* [2,5] ADVP -> ADV NP .
* [2,5] VP -> V NP .
* [1,5] VP -> V ADVP .
* [0,5] S -> NP VP .
```

Extracted Parses

Structure A: (Where "flies" functions as a verb)

Probability: 0.00448 | Cost (-log2): 7.80228

```
(S
  (NP (N time))
  (VP (V flies)
    (ADVP (ADV like)
      (NP (Det an) (N arrow)))))
```

Structure B: (Where "time flies" functions as a noun phrase)

Probability: 0.00096 | Cost (-log2): 10.02467

```
(S
  (NP (N time) (N flies))
  (VP (V like)
    (NP (Det an) (N arrow))))
```

Part 2: Tree Probability Computations

Objective: Use the rule probabilities from the grammar to compute the likelihood of each valid parse.

The sentence is structurally ambiguous, leading to the two derivation trees shown above. The probability for each tree is obtained by multiplying the individual probabilities of every rule applied in the derivation. The cost is represented as the negative base-2 logarithm ($-log_2$) of the total probability.

Analysis of Structure A ("flies" as the main verb)

- **Nested Structure:** [time]$_{NP}$ [flies [like [an arrow]$_{NP}$]$_{ADVP}$]$_{VP}$
- **Rule Composition:** P(S → NP VP) × P(NP → N) × P(N → time) × P(VP → V ADVP) × P(V → flies) × P(ADVP → ADV NP) × P(ADV → like) × P(NP → Det N) × P(Det → an) × P(N → arrow)
- **Calculation:** 1.0 × 0.35 × 0.4 × 0.4 × 0.5 × 1.0 × 1.0 × 0.4 × 1.0 × 0.4 = **0.00448**
- **Overall Cost:** $-log_2(0.00448) \approx$ **7.80229**

Analysis of Structure B ("like" as the main verb)

- **Nested Structure:** [time flies]$_{NP}$ [like [an arrow]$_{NP}$]$_{VP}$
- **Rule Composition:** P(S → NP VP) × P(NP → N N) × P(N → time) × P(N → flies) × P(VP → V NP) × P(V → like) × P(NP → Det N) × P(Det → an) × P(N → arrow)
- **Calculation:** 1.0 × 0.25 × 0.4 × 0.2 × 0.6 × 0.5 × 0.4 × 1.0 × 0.4 = **0.00096**
- **Overall Cost:** $-log_2(0.00096) \approx$ **10.02468**

Observation: The first structure yields a higher probability (and a lower encoding cost), making it roughly 4.67 times more likely under the given grammar rules.

Part 3: Handling Ambiguity with Custom Grammar

Objective: Construct a grammar capable of parsing "the man shot the soldier with a gun" and demonstrably test it.

Proposed Grammar

To properly model prepositional phrase (PP) attachment ambiguity (i.e., whether "with a gun" modifies the action "shot" or the noun "soldier"), the following rules were created. Branching decisions are assigned equal weights (0.5) to reflect the ambiguity seamlessly:

Probability	Production Rule
1.0	S → NP VP
0.5	NP → DT N
0.5	NP → DT N PP
0.5	VP → V NP
0.5	VP → V NP PP
1.0	PP → P NP
0.5	DT → the
0.5	DT → a
0.25	N → man
0.25	N → soldier
0.25	N → gun
1.0	V → shot
1.0	P → with

Chart Output & Ambiguous Parses

The execution accurately captures both syntactic interpretations. Because all branch probabilities are perfectly split, both parse trees evaluate to the same probability (0.000122).

System Chart Output

```
-- Initialization --
[0,0] S -> . NP VP
[0,0] NP -> . DT N
[0,0] NP -> . DT N PP
[0,0] DT -> . the
[0,0] DT -> . a

-- Step 1 (the) --
* [0,1] DT -> the .
[0,1] NP -> DT . N
[0,1] NP -> DT . N PP
[1,1] N -> . man
[1,1] N -> . soldier
[1,1] N -> . gun

-- Step 2 (man) --
* [1,2] N -> man .
* [0,2] NP -> DT N .
[0,2] NP -> DT N . PP
[0,2] S -> NP . VP
[2,2] PP -> . P NP
[2,2] VP -> . V NP
[2,2] VP -> . V NP PP
[2,2] P -> . with
[2,2] V -> . shot

-- Step 3 (shot) --
* [2,3] V -> shot .
[2,3] VP -> V . NP
[2,3] VP -> V . NP PP
[3,3] NP -> . DT N
[3,3] NP -> . DT N PP
[3,3] DT -> . the
```

```
[3,3] DT -> . a

-- Step 4 (the) --
* [3,4] DT -> the .
[3,4] NP -> DT . N
[3,4] NP -> DT . N PP
[4,4] N -> . man
[4,4] N -> . soldier
[4,4] N -> . gun

-- Step 5 (soldier) --
* [4,5] N -> soldier .
* [3,5] NP -> DT N .
[3,5] NP -> DT N . PP
* [2,5] VP -> V NP .
[2,5] VP -> V NP . PP
[5,5] PP -> . P NP
* [0,5] S -> NP VP .
[5,5] P -> . with

-- Step 6 (with) --
* [5,6] P -> with .
[5,6] PP -> P . NP
[6,6] NP -> . DT N
[6,6] NP -> . DT N PP
[6,6] DT -> . the
[6,6] DT -> . a

-- Step 7 (a) --
* [6,7] DT -> a .
[6,7] NP -> DT . N
[6,7] NP -> DT . N PP
[7,7] N -> . man
[7,7] N -> . soldier
[7,7] N -> . gun

-- Step 8 (gun) --
* [7,8] N -> gun .
* [6,8] NP -> DT N .
[6,8] NP -> DT N . PP
* [5,8] PP -> P NP .
[8,8] PP -> . P NP
* [3,8] NP -> DT N PP .
* [2,8] VP -> V NP PP .
[8,8] P -> . with
[2,8] VP -> V NP .
[2,8] VP -> V NP . PP
* [0,8] S -> NP VP .
```

Extracted Trees

Interpretation 1: PP Attached to VP (The man used a gun to shoot)
Probability: 0.0001220711 | Cost (-log2): 13.000000

```
(S
  (NP (DT the) (N man))
  (VP (V shot)
    (NP (DT the) (N soldier))
```

```
    (PP (P with)
      (NP (DT a) (N gun)))))
```

Interpretation 2: PP Attached to NP (The soldier possessed a gun)
Probability: 0.0001220711 | Cost (-log2): 13.000000

```
(S
  (NP (DT the) (N man))
  (VP (V shot)
    (NP (DT the) (N soldier)
      (PP (P with)
        (NP (DT a) (N gun))))))
```

Part 4: Technical Explanations

Objective: Explain specific approaches taken in the submitted parser implementation to guarantee proper functionality and optimal performance.

Ensuring Correctness

To solve the Viterbi parsing problem correctly, the codebase uses a dictionary associated with each chart entry. This structure effectively maintains both the minimum path cost and a comprehensive record of backpointers.

1. **Tracking Weights and Derivations:** Whenever a new chart item is generated, its associated base-2 log cost is calculated. If an equivalent item is rediscovered via a different derivation pathway, the parser compares the newly computed cost with the existing stored cost. The numerical value is overwritten only if the new path provides a strictly lower cost.
2. **Reacting to Alternative Derivations:** Regardless of the cost comparison, the algorithm stores *all* unique backpointers (representing the derivation links) in a list for that item. This ensures that the optimal state's minimum cost is retained while preserving all possible syntactic trees for the final extraction phase.

Ensuring Efficiency

The design carefully controls standard algorithmic bounds, fulfilling the required time and space limits:

1. **Space Bound $O(n^2)$:** The agenda and final charts are structured as columns mapping to input string indices (from 0 to n). Within any column j, an item is identified by its rule and its start index i. Since there are only a finite number of rules in the grammar, there can be at most exactly $O(|G| \times n)$ items per column. Over $n + 1$ columns, the total space is inherently restricted to $O(n^2)$.
2. **Time Bound $O(n^3)$:** In an Earley parser, PREDICT and SCAN operations process items using constant or grammar-bound $O(|G|)$ time. The primary performance driver is the COMPLETE step. When completing an item spanning from index i to index j, the parser searches column i for matching incomplete states. Given that column i possesses at most $O(n)$ entries and there are $O(n^2)$ unique configurations of (i, j) pairs, the completion workload culminates at $O(n^3)$.
3. **Optimal O(1) Enqueuing:** Preventing exponential explosion hinges on validating whether an item already exists before pushing it to the chart. Our implementation realizes columns

as Python dictionaries (which use hash-based lookup). Querying and appending items operates in average-case O(1) time. This fast duplication check categorically prevents identical loops from being incessantly queued and expanded, keeping the execution within expected polynomial bounds.